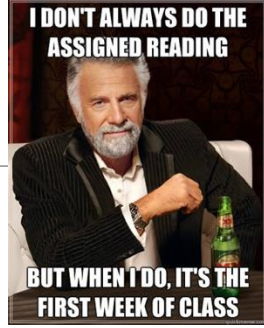
6.100B: Solving Knapsack Problems

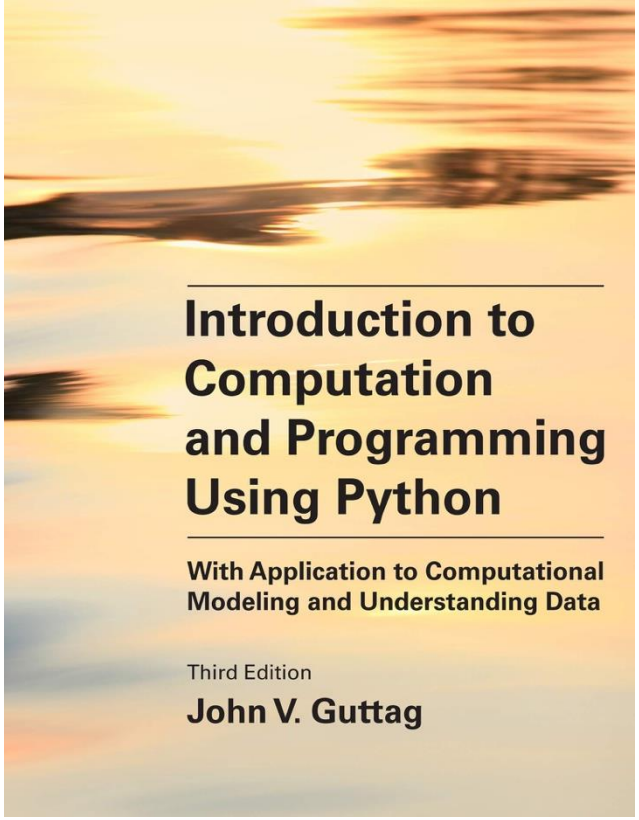
JOHN GUTTAG

MIT EECS

Reading



- Today
  - Sections 14.1-14.2, 15-1
- Next Lecture
  - Chapter 15,2-15.4



All code and an errata sheet at
https://github.com/guttag/Intro-to-Computation-and-Programming

Summarizing Last Lecture

- Optimization problems cover a range of interesting scenarios
  - Characterized by desire to maximize (or minimize) an objective function, subject to a constraint(s) on some parameters of problem
- Writing code to solve them can
  - Provide an answer to a specific question
  - Allow one to compare answers to related questions
- 0/1 knapsack problem describes one broad class of optimization problems
- Brute force approach worked on a small menu

How About a Larger Menu?

Creating a Larger Menu

- When you want to see how things scale, you need large test sets
- You want to generate them, not type them
- Generating them using randomness
  - Helps to avoid bias in test data
  - Can generate different test data each run

Generating Random Values

- Import `random` library
- Many different functions, all based on a deterministic internal function that generates a sequence of pseudo-random numbers
- `random.seed()` initializes that internal function
- With no argument, system initializes using internal system time – hence will generate different sequence of pseudo-random numbers each time
  - With an integer argument, system initializes generation of random numbers from that starting point – hence will generate same sequence of random numbers each time

<u>MUCH</u> more on using randomization in future lectures

Let's Try a Larger Menu

random integers between 1 and max_val

```
def generate_foods(num_foods, max_val, max_cal):
    names = [f'food{n}' for n in range(num_foods)]
    values = [random.randint(1, max_val) for _ in range(num_foods)]
    calories = [random.randint(1, max_cal) for _ in range(num_foods)]
    return names, values, calories
```

```
def test_funcs(func, num_items_list, limit, max_val, max_cost):
    for num_items in num_items_list:
        print(f'Test {func.__name__} for {num_items} items,'
              f' {limit} max')
        random.seed(0)
        names, values, calories = generate_foods(num_items,
                                                 max_val, max_cost)
        foods = build_menu(names, values, calories)
        random.seed()
        random.shuffle(foods)
        solution, value = func(foods, limit)
        print(f'Total value of items taken = {value}')
        for item in solution:
            print(f'    {item}')
```

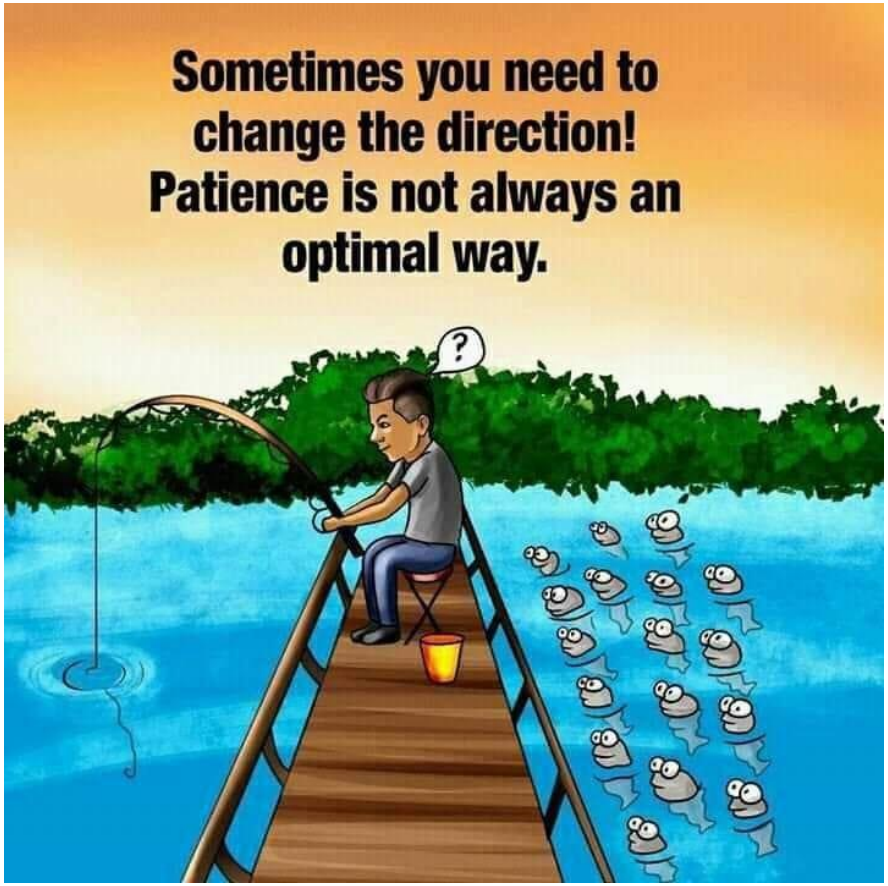
Looking ahead to testing different functions

Creating a parameterized test function

Try It (run code)

```
test_funcs(brute_force, (10, 20, 30), 750, 100, 300)
```

We could keep waiting, but



Results

Test brute_force for 10 items, 750 max
Total value of items taken = 367
food6: <63, 72>
food1: <98, 299>
food5: <66, 145>
food7: <52, 49>
food2: <54, 112>
food4: <34, 72>

Checked 1024 combos

Test brute_force for 20 items, 750 max
Total value of items taken = 508
food17: <97, 267>
food9: <62, 52>
food2: <54, 76>
food13: <65, 105>
food19: <80, 32>
food5: <66, 38>
food4: <34, 51>
food0: <50, 129>

Checked 1,048,576 combos

Test brute_force for 30 items, 750 max

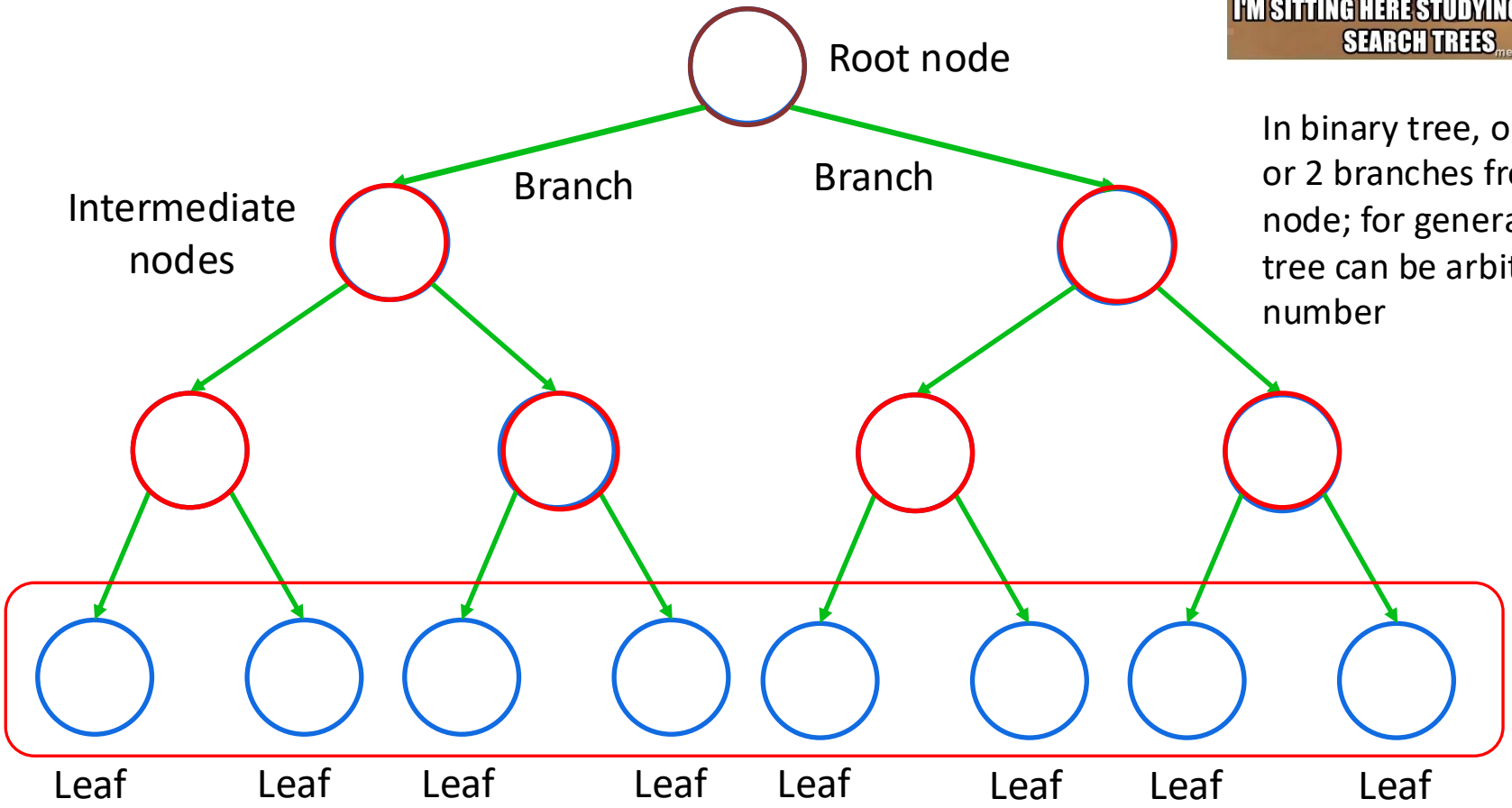
Would check
1,073,741,824 combos

Are We Just Being Stupid?

- Sort of
- 0/1 knapsack problem is inherently **exponential**
- Why? Because the **number of combos** grows that fast
  - $\mathbf{2^n}$ **combinations** for n items
- But don't despair
  - We have options

Decision Trees

Binary Tree Data Structure



In binary tree, only 1 or 2 branches from a node; for general tree can be arbitrary number

Using a Tree to Solve Knapsack Problem

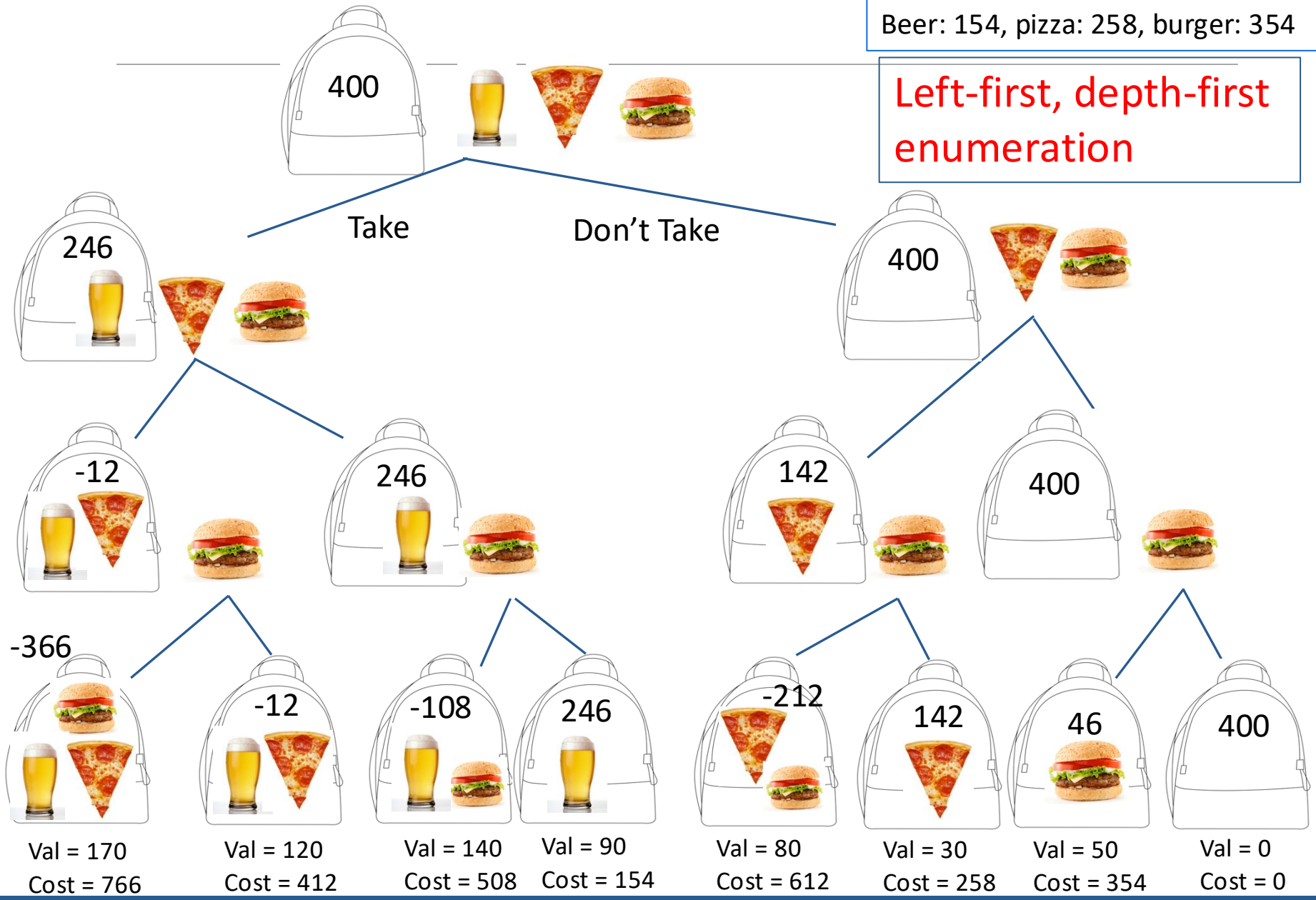
- We can use the tree to generate an exhaustive search
  - Generate all the leaves of the tree
  - Remove any leaves that don't meet constraint
  - Find the best remaining leaf
- Simple example: with calorie budget of 400 calories, chose an optimal set of foods from the menu

Food	beer	pizza	burger
Value	90	30	50
calories	154	258	354

A Tree Enumerates Possibilities

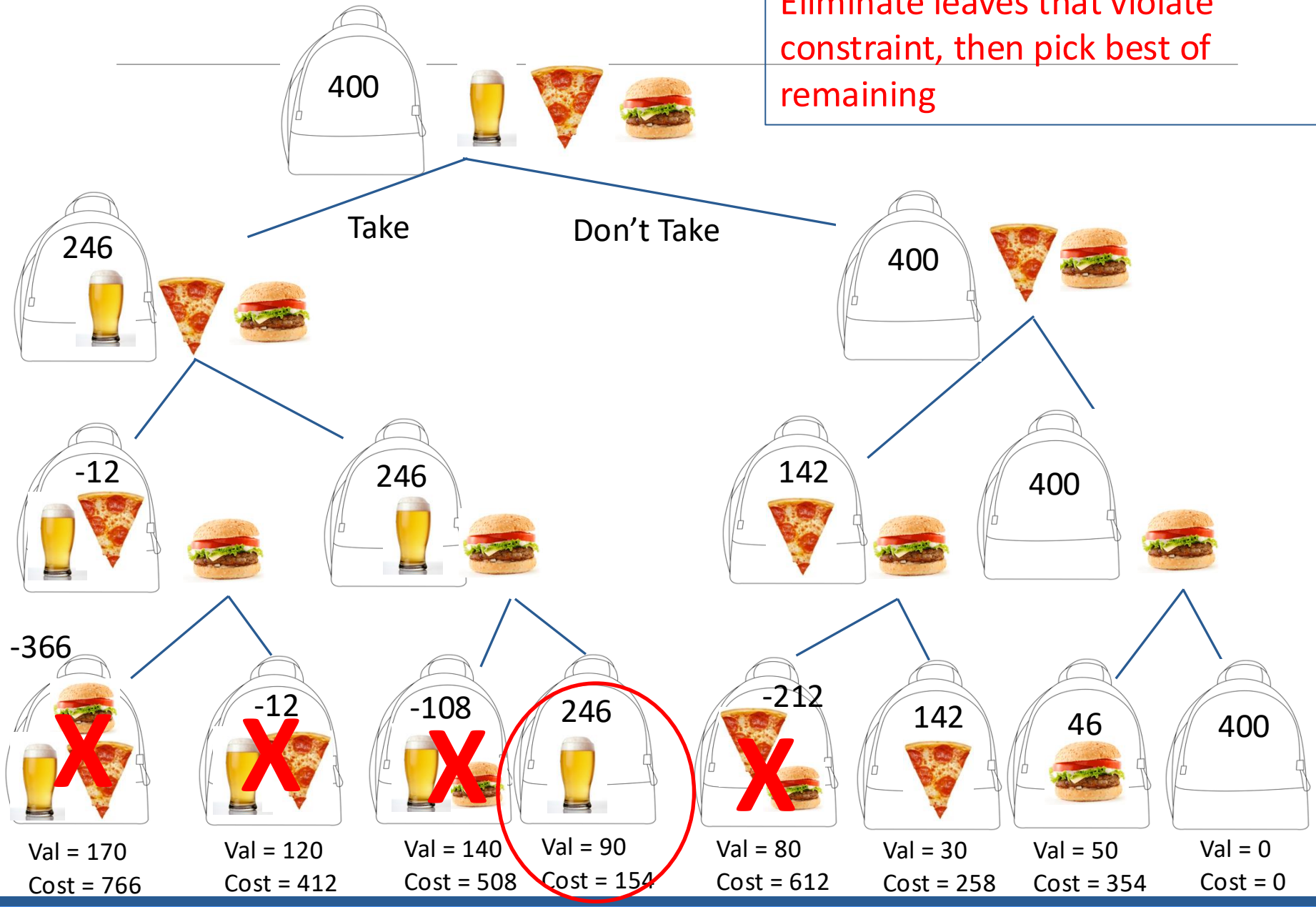
Calories
Beer: 154, pizza: 258, burger: 354

Left-first, depth-first enumeration



A Tree Enumerates Possibilities

Eliminate leaves that violate constraint, then pick best of remaining



We Ought To Be Able To Do Better

- So far, decision tree almost as inefficient as brute force
  - Brute force was dominated by $\boldsymbol{\Theta(n2^n)}$ for generating power set
  - Exploit the fact that solution is encoded along the path from root to leaf
  - Decision tree can bring that down to $\boldsymbol{\Theta(2^n)}$
  - Still prohibitive
- How can we do better?
- As we generate the tree, we can **identify choices that won't work** ***early in the process***
  - We can use this to avoid creating of parts of the tree that cannot have a valid solution

Implementation Plan

- The tree is not actually built
- Paths explored traversed top down starting with the root
- Element is selected from those items still to be considered
  - **If that item fits in the knapsack,** a new node is constructed below the current one that reflects the consequence of choosing to take that item.
  - We also explore the consequences of not taking that item.
- Process is applied recursively to non-leaf children (still have items to consider)
- For any node, best solution is the better of the solutions for each child node
  - For leaf nodes, solution is current value

Not having room for an item means everything along that branch in the tree will **not** be considered – so we don't explore all of the tree

Code

```
def decision_tree(items, capacity):
    """items a list of Item, capacity >= 0
       Solve the 0/1 knapsack problem.
       Return a tuple of an optimal subset of items and their
       total value"""
    # Recursively explore tree
    if not items or capacity == 0: # base case
        return (), 0
    item = items[0] # Get first item not yet seen
    if item.get_cost() > capacity: # Does current item fit
        return decision_tree(items[1:], capacity)
    # Recursively explore consequence of taking current item
    with_item, with_value = decision_tree(items[1:], capacity - item.get_cost()
    with_item += (item,)
    with_value += item.get_value()
    # Recursively explore consequence of not taking current item
    without_item, without_value = decision_tree(items[1:], capacity)
    # Update using better choice
    if with_value > without_value:
        return with_item, with_value
    else:
        return without_item, without_value
```

Try It on Example from Lecture 1

With 750 calories

Food	wine	beer	pizza	burger	fries	coke	apple	donut	cake
Value	89	90	30	50	90	79	90	10	85
Cals	123	154	258	354	365	150	95	195	107

Total value of items taken with brute force = 409

wine: <89, 123>
beer: <90, 154>
pizza: <95, 258>
apple: <50, 95>
cake: <85, 107>

Total value of items taken with decision tree = 409

cake: <85, 107>
apple: <50, 95>
pizza: <95, 258>
beer: <90, 154>
wine: <89, 123>

Gave Us an Optimal Answer

- Finished quickly
- What happens when we have a more extensive menu

Trying It On a Larger Example

```
import time
```

Sometimes we care about “wall time” rather than theoretical complexity

```
start_time = time.time()
test_funcs(brute_force, (20,), 1000, 100, 300)
end_time = time.time()
tot_time = end_time - start_time
print(f'Elapsed time for brute_force = {tot_time:.2f} seconds')

start_time = time.time()
test_funcs(decision_tree, (20,), 1000, 100, 300)
end_time = time.time()
tot_time = end_time - start_time
print(f'Elapsed time for decision_tree = {tot_time:.2f} seconds')
```

Some Results and Observations

Test brute_force for 20 items, 1000 max
Total value of items taken = 584
Elapsed time for brute_force = 2.01 seconds

Test decision_tree for 20 items, 1000 max
Total value of items taken = 584
Elapsed time for decision_tree = 0.04 seconds

- Each finds same answer
- Decision tree considerably faster
- Fast enough?

Some Results and Observations

Test brute_force for 20 items, 1000 max
Total value of items taken = 584
Elapsed time for brute_force = 2.01 seconds

Test decision_tree for 20 items, 1000 max
Total value of items taken = 584
Elapsed time for decision_tree = 0.04 seconds

- Each finds same answer
- Decision tree considerably faster
- Fast enough?

Test decision_tree for 30 items, 1000 max
Total value of items taken = 660
Elapsed time for decision_tree = 1.54 seconds

Test decision_tree for 40 items, 1000 max
Total value of items taken = 785
Elapsed time for decision_tree = 25.55 seconds

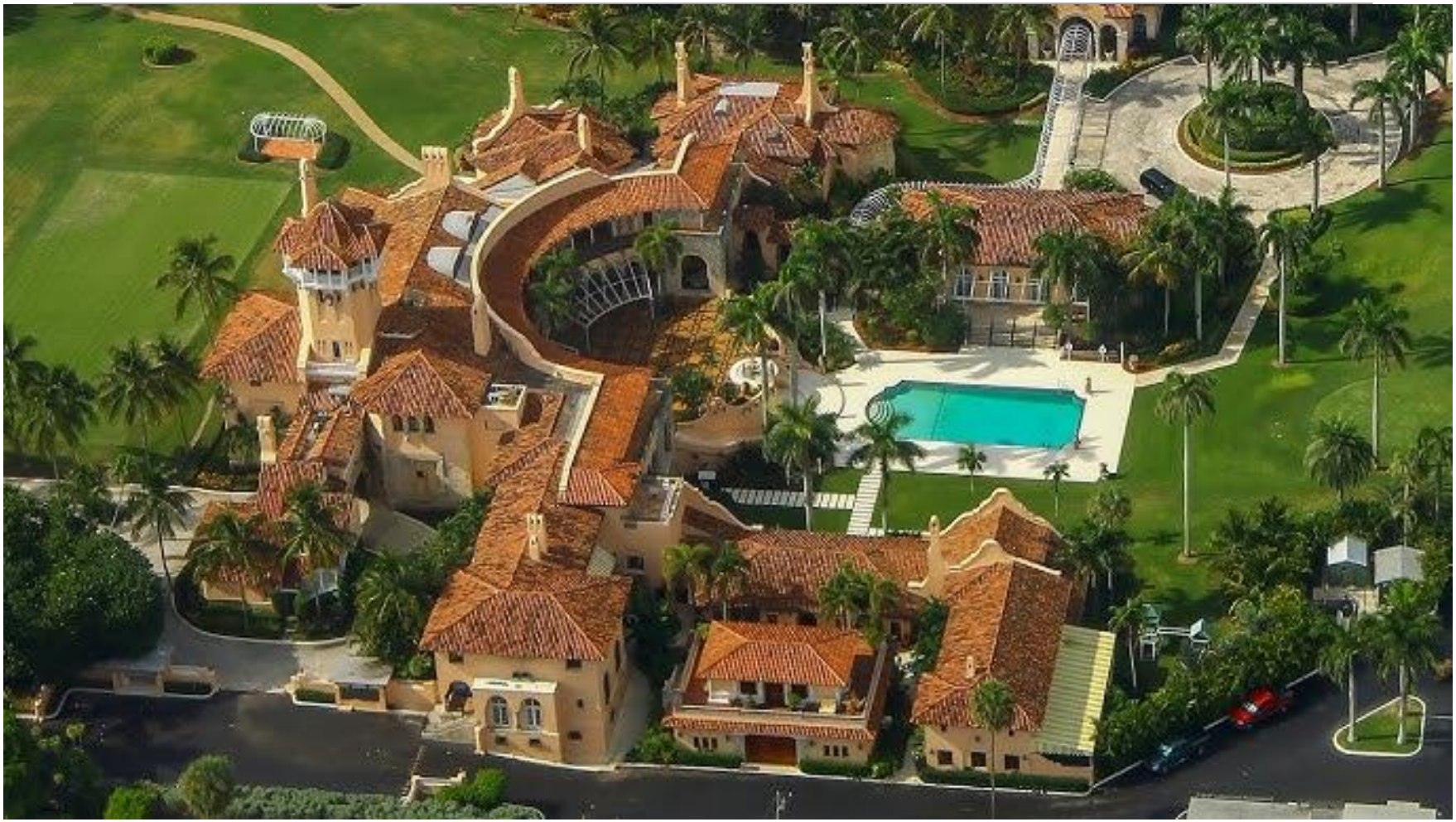
Test decision_tree for 45 items, 1000 max
Total value of items taken = 982
Elapsed time for decision_tree = 697.82 seconds

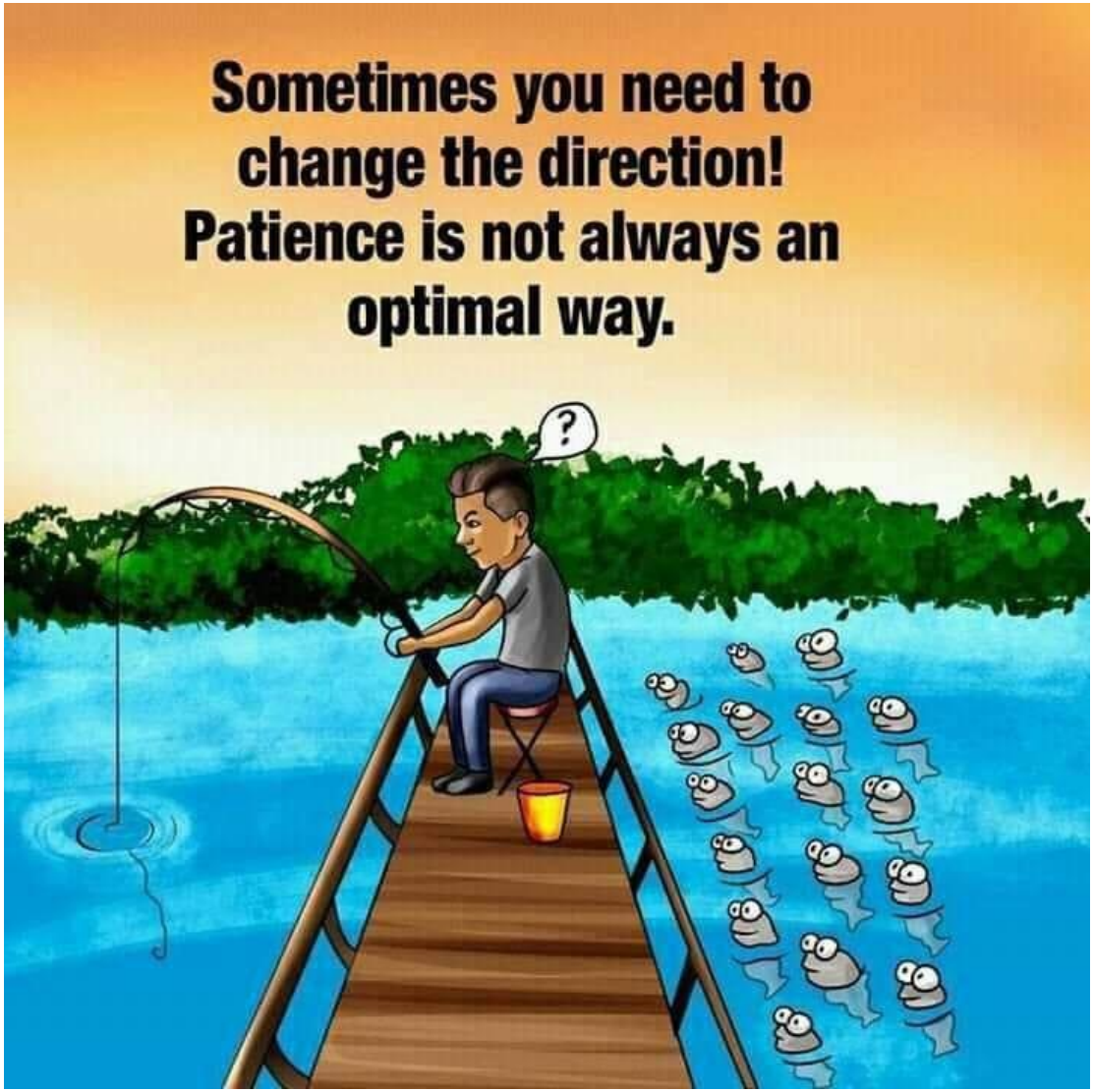
11 minutes might be OK
Trend is alarming
Growth still looks exponential

Is It Fast Enough for a Burglar?

Is It Fast Enough for a Burglar?

Is It Fast Enough for a Burglar?




Sometimes you need to change the direction! Patience is not always an optimal way.
?

“Solving” the 0/1 Knapsack Problem: **GREEDY STRATEGY**

Greedy strategy in a nutshell

- **Repeatedly:**
  - Choose the **"best"** remaining item
  - Stuff it in the knapsack only if it will fit

- But what does "best" mean?
  - Most valuable?
  - Lightest weight (or lowest cost)?
  - Highest value per unit weight?
  - Something else?

Turning Idea Into Code

```
def knapsack_greedy(items, capacity):
    """items a list of Item, capacity a non-negative number
    Uses a greedy algorithm to provide an approximation of
    a solution to the 0/1 knapsack problem.
    Returns a list of items and the value of that list."""
    items_sorted = sorted(items,
                          key=lambda item: item.get_density(),
                          reverse=True)
    knapsack = []
    total_value = 0
    for item in items_sorted:
        if item.get_cost() <= capacity:
            knapsack.append(item)
            capacity -= item.get_cost()
            total_value += item.get_value()
    return knapsack, total_value
```

Choice for “best” item

Don’t exceed constraint on total cost

Decrease capacity

Increase accumulated value

Compare to Decision Tree

```
start_time = time.time()
test_funcs(decision_tree, (40,), 1000, 100, 300)
end_time = time.time()
tot_time = end_time - start_time
print(f'Elapsed time for decision_tree = {tot_time:.2f} seconds')

start_time = time.time()
test_funcs(knapsack_greedy, (40,), 1000, 100, 300)
end_time = time.time()
tot_time = end_time - start_time
print(f'Elapsed time for knapsack_greedy = {tot_time:.2f} seconds')
```

Test decision_tree for 40 items, 1000 max
Total value of items taken = 785
Elapsed time for decision_tree = 16.4 seconds

Good news: really fast

Test knapsack_greedy for 40 items, 1000 max
Total value of items taken = 775
Elapsed time for knapsack_greedy = 0.00 seconds

Bad news: “wrong” answer

Perhaps It's the Wrong Metric for Best

```
def knapsack_greedy(items, capacity):
    """items a list of Item, capacity a non-negative number
    Uses a greedy algorithm to provide an approximation of
    a solution to the 0/1 knapsack problem.
    Returns a list of items and the value of that list."""
    items_sorted = sorted(items, key=lambda item: item.get_density(),
                          reverse=True)
    knapsack = []
    total_value = 0
    for item in items_sorted:
        if item.get_cost() <= capacity:
            knapsack.append(item)
            capacity -= item.get_cost()
            total_value += item.get_value()
    return knapsack, total_value
```

Hard-wired

More General Version

```
def knapsack_greedy(items, capacity,
                    key=lambda x: x.get_density()):
    """items a list of item, capacity a non-negative number
    Uses a greedy algorithm to provide an approximation of
    a solution to the 0/1 knapsack problem.
    Returns a list of items and the value of that list."""
    items_copy = sorted(items, key=key, reverse=True)
    result = []
    total_value, total_weight = 0, 0
    for item in items_copy:
        if total_weight + item.get_cost() <= capacity:
            result.append(item)
            total_weight += item.get_cost()
            total_value += item.get_value()
    return result, total_value
```

Use this to set priority when choosing items

Code to Test It

```
def test_greedies(num_items_list, limit, compare = False):
    for num_items in num_items_list:
        names, values, calories = generate_foods(num_items, 100, 300)
        foods = build_menu(names, values, calories)
        if compare:
            optimum = decision_tree(foods, limit)[1]
            print(f'Optimal value = {optimum}')
        metrics = {'value': Item.get_value,
                   'cost': lambda x: float('inf') if x.get_cost() == 0
                                     else 1 / x.get_cost(),
                   'density': Item.get_density}
        for key in metrics:
            print(f'Use key {key}')
            func = lambda foods, limit: knapsack_greedy(foods, limit,
                                                        metrics[key])
            solution, value = func(foods, limit)
            print(f'Total value of items taken = {value}')
            for item in solution:
                print(f'   {item}')
```

```
random.seed(1)
test_greedies((10,), 750, compare = True)
```

Results

Optimal value = 452

Use key value
Total value of items taken = 452
food2: <98, 49>
food7: <98, 2>
food1: <73, 108>
food6: <64, 222>
food9: <61, 137>
food8: <58, 229>

Use key cost
Total value of items taken = 397
food7: <98, 2>
food4: <33, 15>
food2: <98, 49>
food1: <73, 108>
food9: <61, 137>
food0: <18, 195>
food5: <16, 200>

Use key density
Total value of items taken = 445
food7: <98, 2>
food4: <33, 15>
food2: <98, 49>
food1: <73, 108>
food9: <61, 137>
food6: <64, 222>
food0: <18, 195>

For key = value, result is optimal

Always true?

Try a Different Seed (2)

Beauty of way we generate data

Optimal value = 272

Use key value
Total value of items taken = 271
  food5: <95, 202>
  food6: <86, 261>
  food9: <78, 228>
  food1: <12, 19>
Use key cost
Total value of items taken = 202
  food1: <12, 19>
  food3: <47, 82>
  food0: <8, 109>
  food7: <40, 191>
  food5: <95, 202>

Use key density
Total value of items taken = 272
  food1: <12, 19>
  food3: <47, 82>
  food5: <95, 202>
  food9: <78, 228>
  food7: <40, 191>

This time density achieves optimum

Try a Different Seed (3)

Optimal value = 316

Use key value
Total value of items taken = 305
  food7: <81, 277>
  food5: <78, 99>
  food1: <76, 241>
  food2: <70, 133>
Use key cost
Total value of items taken = 303
  food0: <31, 7>
  food5: <78, 99>
  food4: <48, 120>
  food2: <70, 133>
  food1: <76, 241>

Use key density
Total value of items taken = 303
  food0: <31, 7>
  food5: <78, 99>
  food2: <70, 133>
  food4: <48, 120>
  food1: <76, 241>

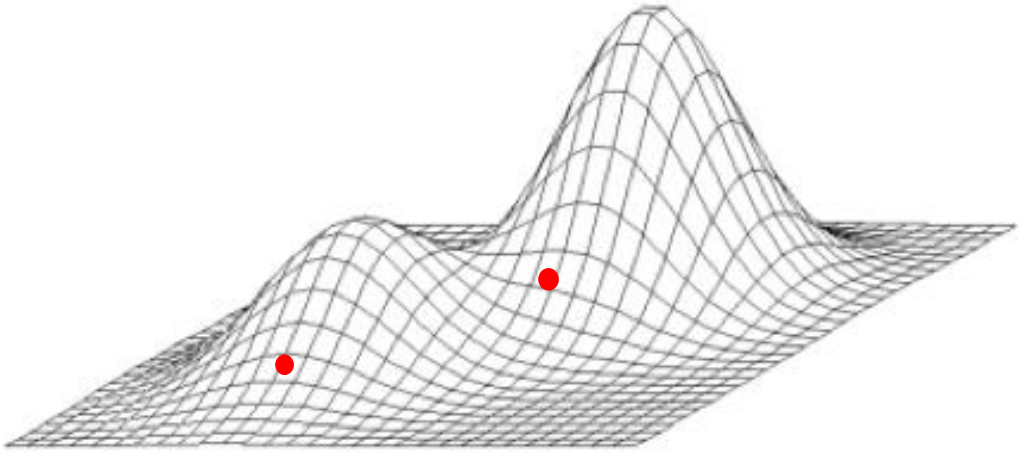
This time none finds optimum

Why Different Answers?

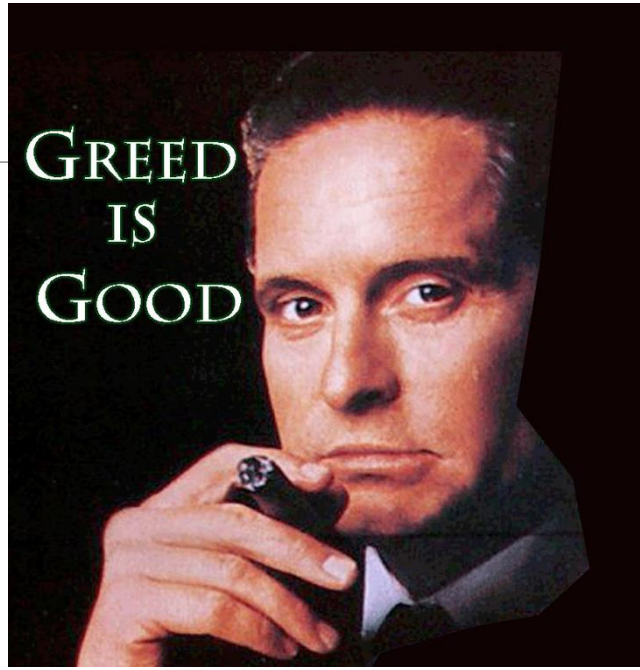
- Different metrics lead to **different priority orders**
- While always trying to optimize value, order in which we consider items may be different
- Recall that we **can't backtrack**
  - Once we decide on including an item, can't later decide that some combination of lower valued items is actually better than taking that item

A Problem with Greedy Algorithms

- **Sequence of locally "optimal" choices** don't always yield a globally optimal solution
  - Consider a surface
  - Local optimization is "hill climbing" – a greedy choice is equivalent to walking up the steepest slope along a dimension corresponding to this item until reach a peak
  - **Finding the best solution is possible, but not guaranteed**



The Pros of Greedy



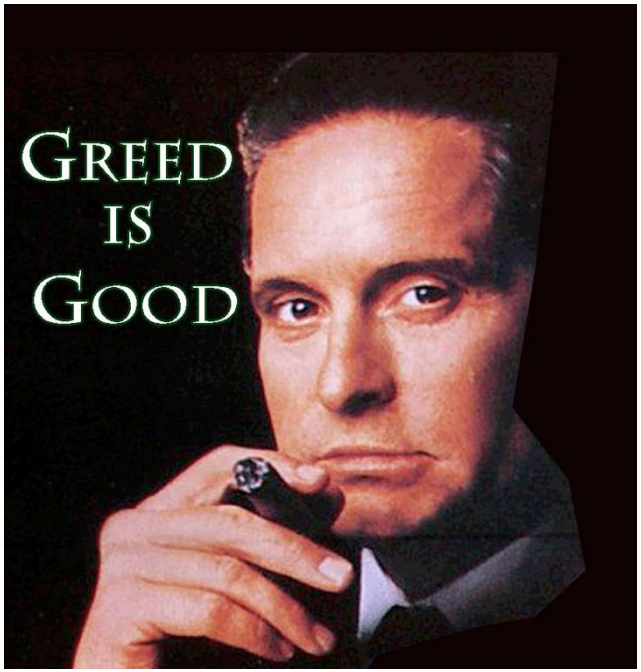
- Easy to implement
- Computationally efficient
  - Log linear versus exponential
  - Some problems could be solved by greedy algorithm that are not feasible by brute force
- How efficient?

```
num_items = 100_000
start_time = time.time()
test_funcs(knapsack_greedy, (num_items,), 10_000, 100, 300)
end_time = time.time()
tot_time = end_time - start_time
print(f'Elapsed time for knapsack_greedy with {num_items:,} = '
      f'{tot_time:.2f} seconds')
```

Run it

Your Turn

- Who is this guy, and from what movie?



Which Movie?

Main Street
Wall Street
Other People's Money
The Art of the Deal
The Big Short

"Greed, for lack of a better word, is good."
Gordon Gekko.

Gordon Gekko Modeled On Ivan Boesky

"Greed is all right, by the way. I want you to know that. I think greed is healthy. You can be greedy and still feel good about yourself."

Ivan Boesky, May 18, 1986 at UC-Berkeley's Haas School of Business Commencement

Boesky indicted later that year for insider trading

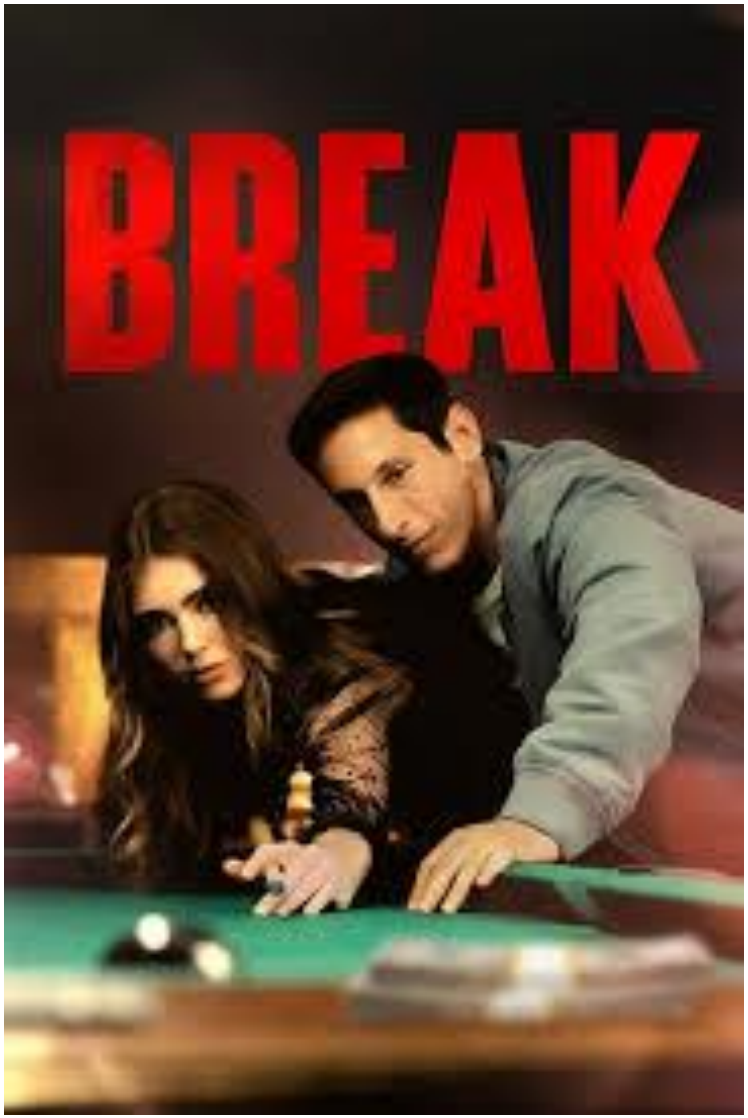
Pleaded guilty to conspiracy in 1987
Sentenced to 3 years in prison

The prosecutor

The Other Con of Greedy

- Does **not** always yield the best solution
  - **Don't even know how good the approximation is**
  - Not clear what criterion we should use for greedy – different choices lead to different solutions
- Suppose we want to find a truly optimal solution, but can't afford to use exhaustive search?

Five Minute Break

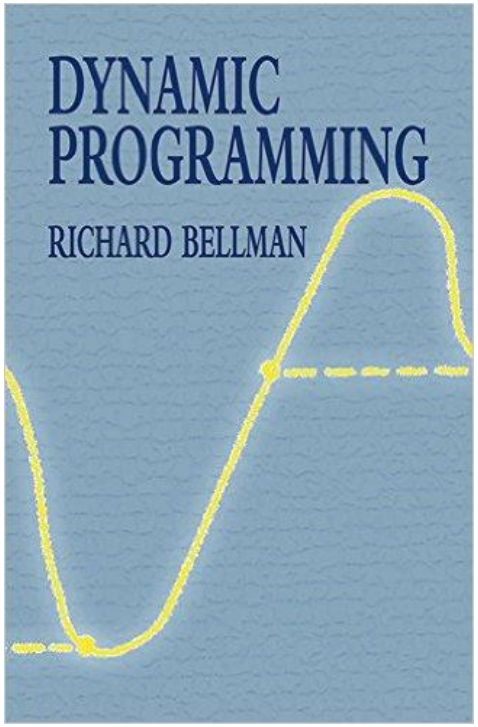


Summary, So Far

- Optimization problems cover a range of interesting scenarios
- 0/1 knapsack problem describes one broad class of optimization problems
  - Computational complexity typical of many optimization problems
- Brute force approaches often far too expensive
- Greedy algorithms can provide reasonable (but not optimal) solutions in many cases, at low computational cost
- Must we choose between efficiency and optimality?

It It Hopeless?

- In theory, yes
- In practice, no!
- Dynamic programming to the rescue



Fibonacci Numbers

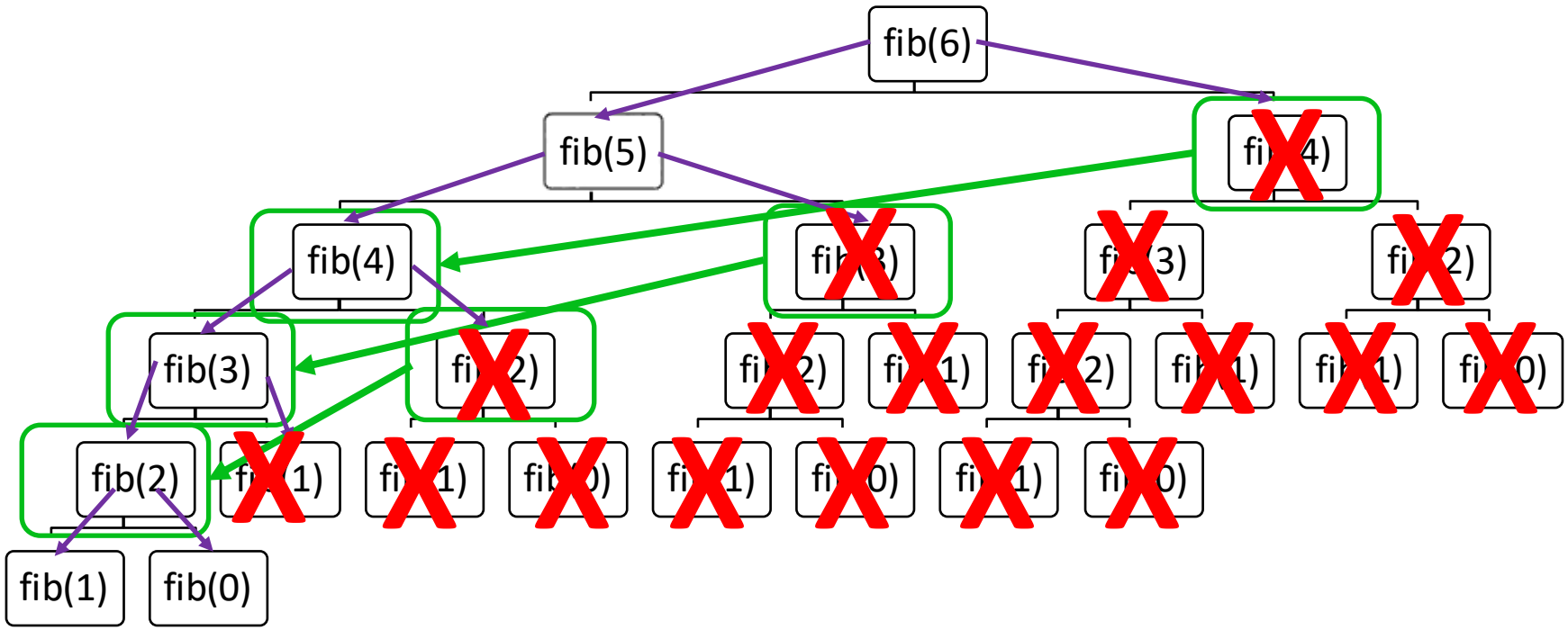
```
def fib(n):
    if n == 0 or n == 1:
        return 1
    else:
        return fib(n-1) + fib(n-2)


for i in range(0, 2001):
    print(f"Try fib({i}) = {fib(i):,}")
```

How far do think we'll get before our patience runs out?

Why is it taking so long?

Call Tree for Recursive Fibonnaci(6) = 13



Order of recursive calls

Don't need to compute these, as we already know the value of each call

Clearly a Bad Idea to Repeat Work

The reward for work well done is the opportunity to do more.

Jonas Salk

- Trade time for space
- Create a **table** to record what we've done
  - Before computing fib(n), **check if value of fib(n) already stored** in the table
  - If so, look it up
  - If not, compute it and then add it to table
- Called **memoization**

Using a Memo to Compute Fibonacci

```
def fast_fib(n, memo=None):
    if memo == None:
        memo = {}
    if n == 0 or n == 1:
        return 1
    try:
        return memo[n]
    except KeyError:
        result = fast_fib(n-1, memo) + fast_fib(n-2, memo)
        memo[n] = result
        return result
```

If already know value, just return from dictionary

Otherwise, compute value, store for later use, and return result

Note use of exceptions as a control flow mechanism

Let's Try It

```
for i in range(0, 2001, 100):
    print(f'Try fib({i}) = {fast_fib(i):,}')
```

Let's run it

How far do think we'll get before our patience runs out?

Tabular Approach to DP

- **Memoization is top-down**
  - Start from problem to be solved, the biggest problem
  - **Add entries** to memo **as you solve** new sub-problems
- **Tabular is bottom-up**
  - **Pre-allocate memo table** for all possible sub-problems
  - Populate the table systematically, from smallest problem

```
def fast_fib_tab(n):
    # Allocate table for indicies 0 through n
    # Pre-populate with values for fib(0) and fib(1)
    tab = [1] * (n+1)
    for i in range(2, n+1):
        tab[i] = tab[i-1] + tab[i-2]
    return tab[n]
```

This will generate all values for Fibonacci, starting from smallest

Tabularization vs. Memoization

- If the original problem requires all subproblems to be solved, tabular method usually better
  - Tabular method easier to implement
  - Tabulation usually faster – it has **no overhead for recursion** and can use a pre-allocated fixed size list
- If only some of the subproblems need to be solved to solve the original problem, memoization usually better
  - More efficient because subproblems are solved **lazily**, only perform the computations that are needed

When Does DP Help?

- Dynamic programming will help if a problem has the following two characteristics
  - **Optimal substructure**: a **globally optimal solution** can be found by combining **optimal solutions to local subproblems**
    - For x > 1, fib(x) = fib(x-1) + fib(x-2)
  - **Overlapping subproblems**: finding an optimal solution involves solving the **same subproblem multiple times**
    - Compute fib(x) many times
- Without overlapping subproblems, it will still find optimal answer, but no faster than brute force

Why It Is Called Dynamic Programming?

"The 1950s were not good years for mathematical research… I felt I had to do something to shield Wilson and the Air Force from the fact that I was really doing mathematics... What title, what name, could I choose?

... It's impossible to use the word dynamic in a pejorative sense. Try thinking of some combination that will possibly give it a pejorative meaning. It's impossible.

Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to. So, I used it as an umbrella for my activities."

Richard Bellman, *Eye of the Hurricane: an Autobiography*

Stop and Think

- Which of the following have both optimal substructure and overlapping subproblems

1. Computing Fibonacci numbers
2. Merge sort — Doesn't have overlapping subproblems as never merge same lists twice
3. Ternary search
4. Recursive factorial — Doesn't have overlapping subproblems
5. Spelling correction

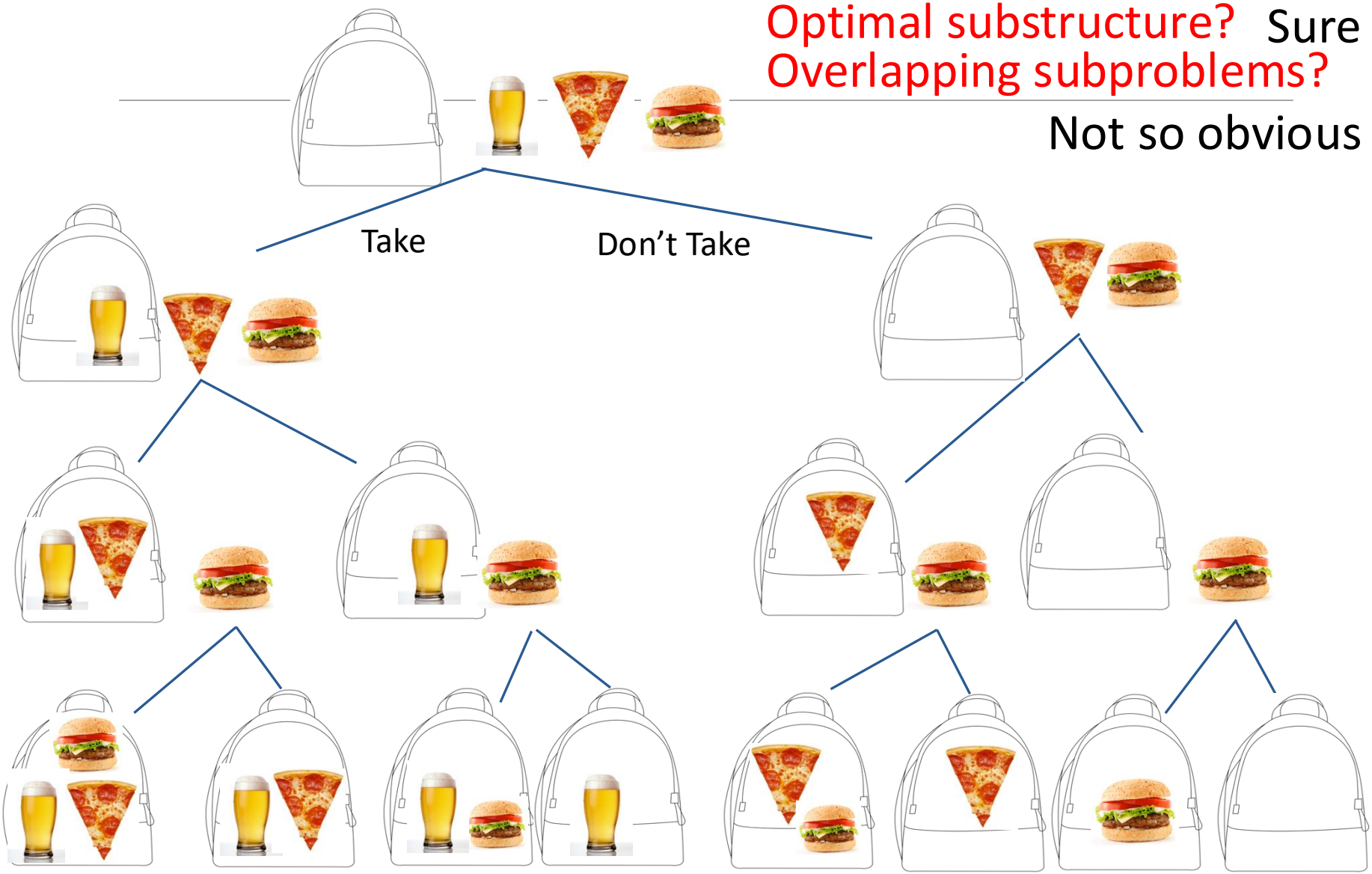
What About 0/1 Knapsack Problem?

Does the Knapsack Problem Exhibit?

- **Optimal substructure:** a globally optimal solution can be found by combining optimal solutions to local subproblems

- **Overlapping subproblems:** finding an optimal solution involves solving the same problem multiple times

Optimal substructure? Sure
Overlapping subproblems?
Not so obvious



Val = 170
Cost = 766

Val = 120
Cost = 412

Val = 140
Cost = 508

Val = 90
Cost = 145

Val = 80
Cost = 612

Val = 30
Cost = 258

Val = 50
Cost = 354

Val = 0
Cost = 0

A Different Item Set

Item	A	B	C	D	E	F	G	H	I
Value	89	90	10	100	90	79	50	10	85
Cost	123	154	31	354	365	150	95	195	107

Consider two nodes in decision tree:

1: Take B and not A or C

2. Take A and C, but not B

Have same problem to solve in both cases:

Given a remaining budget of (initial capacity – 154), what is the optimal subset of items D – I?

Problem Being Solved at Each Node

- Given remaining capacity, maximize value by choosing among remaining items
- Set of previously chosen items, or even value of that set, doesn't matter!
  - Just need to consider cost remaining and items remaining
- So, let's give DP a shot
- In the next lecture